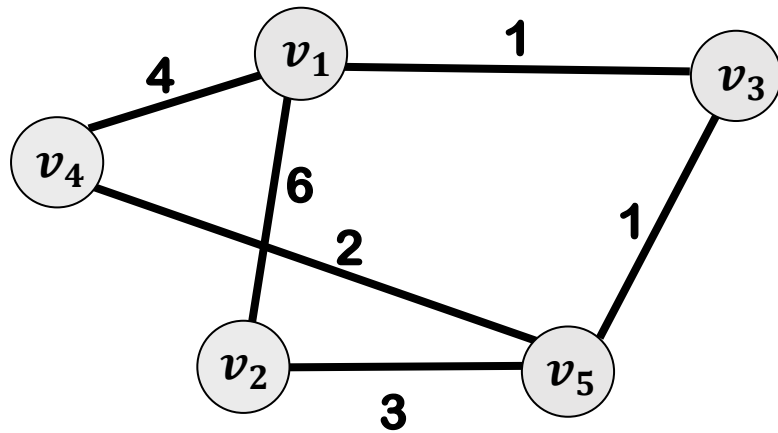


Greedy

Practice problems

Kruskal: Example



	v_1	v_2	v_3	v_4	v_5
v_1	0	6	1	4	∞
v_2	6	0	∞	∞	3
v_3	1	∞	0	∞	1
v_4	4	∞	∞	0	2
v_5	∞	3	1	2	0

W

Kruskal: Example

```
Kruskal(E)
  E.sort
  F=[]
  FOR k=1 to n DO    S[k]=k
  WHILE len(F)!=n-1 DO
    e=(vi,vj)=min_length_edge
    IF S[i]!=S[j] THEN
      F.add(e)
      Min=min(S[i],S[j])
      Max=max(S[i],S[j])
      FOR k=1 to n DO
        IF S[k]==Max THEN S[k]=Min
      E.del(e)
  RETURN F
```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

w

E.sort()

F=[]

	v_1	v_2	v_3	v_4	v_5
v_1	0	6	1	4	∞
v_2	6	0	∞	∞	3
v_3	1	∞	0	∞	1
v_4	4	∞	∞	0	2
v_5	∞	3	1	2	0

W

$F = []$

$E = [v_1 v_3, v_3 v_5, v_4 v_5, v_2 v_5, v_1 v_4, v_1 v_2]$

FOR k=1 to n DO S[k]=k

	v_1	v_2	v_3	v_4	v_5
v_1	0	6	1	4	∞
v_2	6	0	∞	∞	3
v_3	1	∞	0	∞	1
v_4	4	∞	∞	0	2
v_5	∞	3	1	2	0

W

S

1	2	3	4	5
1	2	3	4	5

$F = []$

$E = [v_1 v_3, v_3 v_5, v_4 v_5, v_2 v_5, v_1 v_4, v_1 v_2]$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	2	3	4	5

$F = []$

$E = [v_1v_3, v_3v_5, v_4v_5, v_2v_5, v_1v_4, v_1v_2]$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	2	1	4	5

$$F = [v_1 v_3]$$

$$E = [\cancel{v_1 v_3}, v_3 v_5, v_4 v_5, v_2 v_5, v_1 v_4, v_1 v_2]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	2	1	4	5

$$F = [v_1 v_3]$$

$$E = [\cancel{v_1 v_3}, v_3 v_5, v_4 v_5, v_2 v_5, v_1 v_4, v_1 v_2]$$


```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	2	1	4	5

$$F = [v_1 v_3]$$

$$E = [\cancel{v_1 v_3}, v_3 v_5, v_4 v_5, v_2 v_5, v_1 v_4, v_1 v_2]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	2	1	4	1

$$F = [v_1 v_3, v_3 v_5]$$

$$E = [\cancel{v_1 v_3}, \cancel{v_3 v_5}, v_4 v_5, v_2 v_5, v_1 v_4, v_1 v_2]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	2	1	4	1

$$F = [v_1 v_3, v_3 v_5]$$

$$E = [\cancel{v_1 v_3}, \cancel{v_3 v_5}, v_4 v_5, v_2 v_5, v_1 v_4, v_1 v_2]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	2	1	4	1

$$F = [v_1 v_3, v_3 v_5]$$

$$E = [\cancel{v_1 v_3}, \cancel{v_3 v_5}, v_4 v_5, v_2 v_5, v_1 v_4, v_1 v_2]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	2	1	1	1

$$F = [v_1 v_3, v_3 v_5, v_4 v_5]$$

$$E = [\cancel{v_1 v_3}, \cancel{v_3 v_5}, \cancel{v_4 v_5}, v_2 v_5, v_1 v_4, v_1 v_2]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	2	1	1	1

$$F = [v_1 v_3, v_3 v_5, v_4 v_5]$$

$$E = [\cancel{v_1 v_3}, \cancel{v_3 v_5}, \cancel{v_4 v_5}, v_2 v_5, v_1 v_4, v_1 v_2]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	2	1	1	1

$$F = [v_1 v_3, v_3 v_5, v_4 v_5]$$

$$E = [\cancel{v_1 v_3}, \cancel{v_2 v_5}, \cancel{v_4 v_5}, v_2 v_5, v_1 v_4, v_1 v_2]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	1	1	1	1

$$F = [v_1 v_3, v_3 v_5, v_4 v_5, v_2 v_5]$$

$$E = [\cancel{v_1 v_3}, \cancel{v_3 v_5}, \cancel{v_4 v_5}, \cancel{v_2 v_5}, v_1 v_4, v_1 v_2]$$

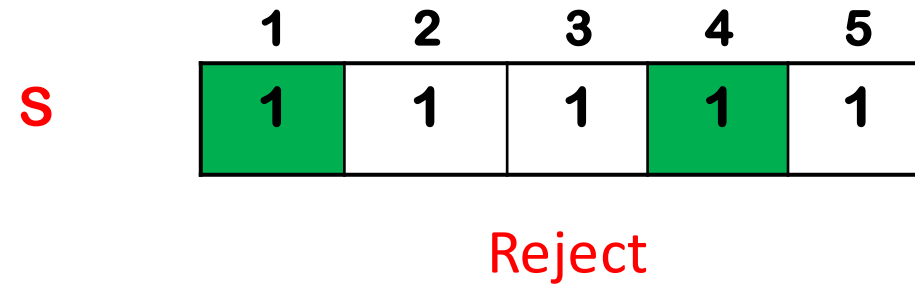

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

w



$$F = [v_1 v_3, v_3 v_5, v_4 v_5, v_2 v_5]$$

$$E = [\cancel{v_1 v_3}, \cancel{v_3 v_5}, \cancel{v_4 v_5}, \cancel{v_2 v_5}, v_1 v_4, v_1 v_2]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	1	1	1	1

$$F = [v_1 v_3, v_3 v_5, v_4 v_5, v_2 v_5]$$

$$E = [\cancel{v_1 v_3}, \cancel{v_3 v_5}, \cancel{v_4 v_5}, \cancel{v_2 v_5}, \cancel{v_1 v_4}, v_1 v_2]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	1	1	1	1

$$F = [v_1 v_3, v_3 v_5, v_4 v_5, v_2 v_5]$$

$$E = [\cancel{v_1 v_3}, \cancel{v_3 v_5}, \cancel{v_4 v_5}, \cancel{v_2 v_5}, \cancel{v_1 v_4}, v_1 v_2]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	1	1	1	1

Reject

$$F = [v_1 v_3, v_3 v_5, v_4 v_5, v_2 v_5]$$

$$E = [\cancel{v_1 v_3}, \cancel{v_3 v_5}, \cancel{v_4 v_5}, \cancel{v_1 v_4}, \cancel{v_1 v_5}, v_1 v_2]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

1	2	3	4	5
1	1	1	1	1

$$F = [v_1v_3, v_3v_5, v_4v_5, v_2v_5]$$

$$E = [\cancel{v_1v_3}, \cancel{v_3v_5}, \cancel{v_4v_5}, \cancel{v_2v_5}, \cancel{v_1v_4}, \cancel{v_1v_2}]$$

```

e=(vi,vj)=min_length_edge
IF S[i]≠S[j] THEN
  F.add(e)
  Min, Max=min(S[i],S[j]), max(S[i],S[j])
  FOR k=1 to n DO
    IF S[k]==Max THEN S[k]=Min

```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	6	1	4	∞
v ₂	6	0	∞	∞	3
v ₃	1	∞	0	∞	1
v ₄	4	∞	∞	0	2
v ₅	∞	3	1	2	0

W

S

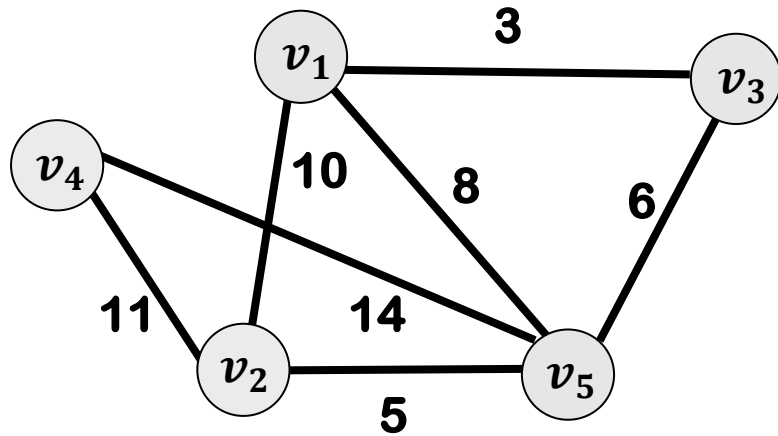
1	2	3	4	5
1	1	1	1	1

|F|=n-1

$$F = [v_1v_3, v_3v_5, v_4v_5, v_2v_5]$$

$$E = [\cancel{v_1v_3}, \cancel{v_3v_5}, \cancel{v_4v_5}, \cancel{v_2v_5}, \cancel{v_1v_4}, \cancel{v_1v_2}]$$

Prim: Example



	v_1	v_2	v_3	v_4	v_5
v_1	0	10	3	∞	8
v_2	10	0	∞	11	5
v_3	3	∞	0	∞	6
v_4	∞	11	∞	0	14
v_5	8	5	6	14	0

W

Prim: Example

```
Prim(W)
F=∅
FOR i=2 TO n DO
    nearest[i]=1
    distance[i]=W[1][i]
FOR j=1 TO n-1 DO
    b=index of minimum element in distance
    a=nearest[b]
    F.add((a,b))
    distance.delete(b)
    nearest.delete(b)
    FOR EACH i in distance DO
        IF W[i][b]<distance[i] THEN
            distance[i]=W[i][b]
            nearest[i]=b
RETURN F
```

	v ₁	v ₂	v ₃	v ₄	v ₅
v ₁	0	10	3	∞	8
v ₂	10	0	∞	11	5
v ₃	3	∞	0	∞	6
v ₄	∞	11	∞	0	14
v ₅	8	5	6	14	0

W

Prim: Example

	v_1	v_2	v_3	v_4	v_5
v_1	0	10	3	∞	8
v_2	10	0	∞	11	5
v_3	3	∞	0	∞	6
v_4	∞	11	∞	0	14
v_5	8	5	6	14	0

W

nearest

1	2	3	4	5
-	-	-	-	-

distance

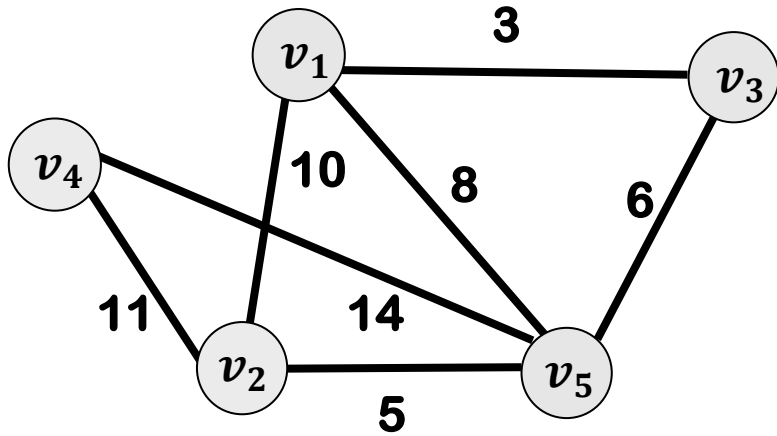
1	2	3	4	5
-	-	-	-	-

```

FOR i=2 TO n DO
    nearest[i]=1
    distance[i]=W[1][i]

```

vertices	1	2	3	4	5
nearest	-	1	1	1	1
distance	-	10	3	∞	8



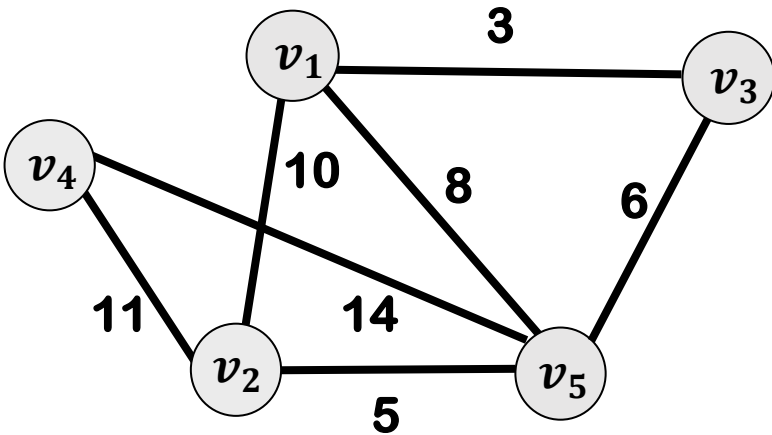
```

b=index of minimum element in distance
a=nearest[b]
F.add((a,b))

```

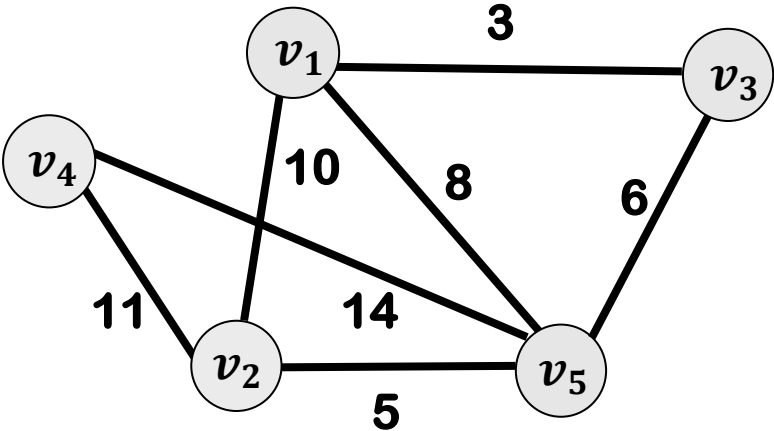
vertices	1	2	3	4	5
nearest	-	1	1	1	1
distance	-	10	3	∞	8

$b=3$
 $a=1$
 $F=\{(3,1)\}$



`distance.delete(b)`
`nearest.delete(b)`

vertices	1	2	3	4	5
nearest	-	1	-	1	1
distance	-	10	-	∞	8



```

if(W[2][3] < distance[2]):
    distance[2] = W[2][3]
    nearest[2] = 3

```

```

if(W[4][3] < distance[4]):
    distance[4] = W[4][3]
    nearest[4] = 3

```

```

if(W[5][3] < distance[5]):
    distance[5] = W[5][3]
    nearest[5] = 3

```

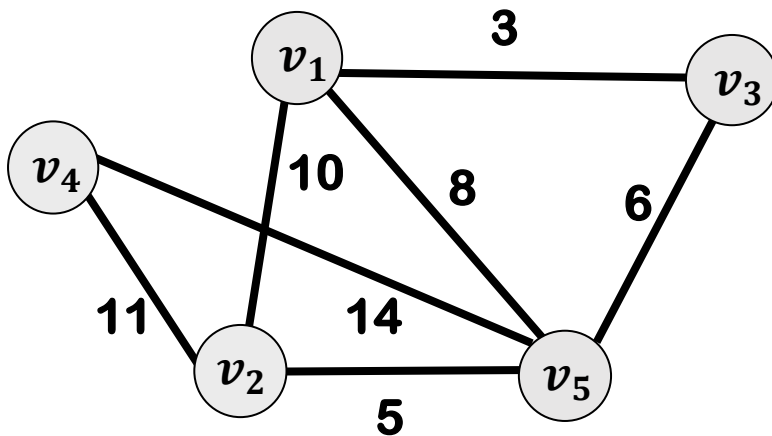
FOR EACH i in distance DO

```

    IF  $W[i][b] < \text{distance}[i]$  THEN
         $\text{distance}[i] = W[i][b]$ 
         $\text{nearest}[i] = b$ 

```

vertices	1	2	3	4	5
nearest	-	1	-	1	1
distance	-	10	-	∞	8
nearest	-	1	-	1	3
distance	-	10	-	∞	6



```

b=index of minimum element in distance
a=nearest[b]
F.add((a,b))

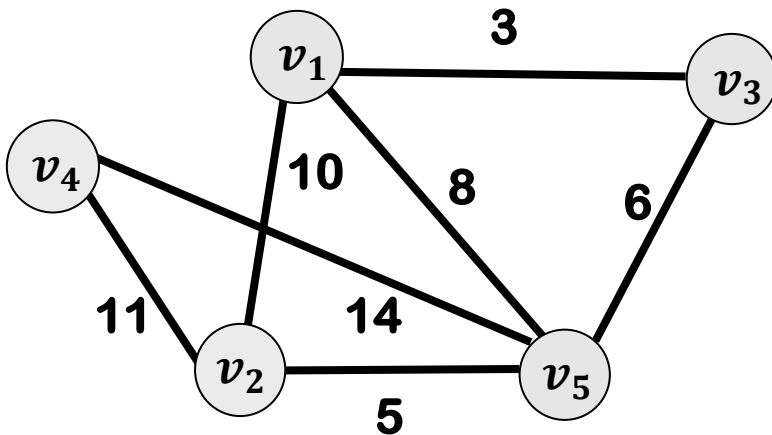
```

vertices	1	2	3	4	5
nearest	-	1	-	1	1
distance	-	10	-	∞	8
nearest	-	1	-	1	3
distance	-	10	-	∞	6

b=5

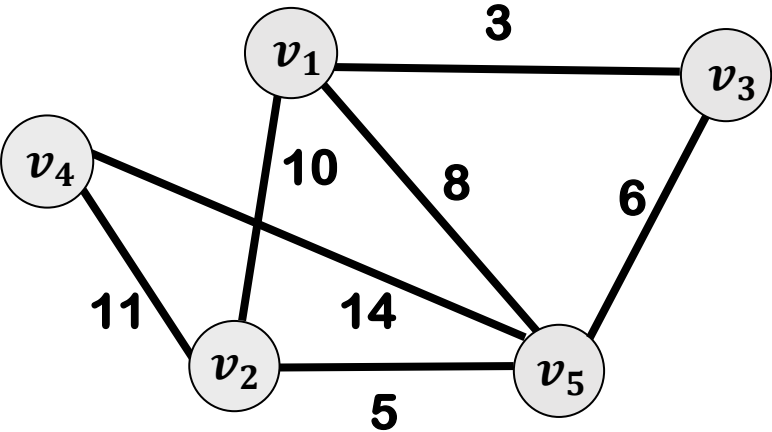
a=3

$F=\{(3,1),(3,5)\}$



distance.delete(b)
nearest.delete(b)

vertices	1	2	3	4	5
nearest	-	1	-	1	-
distance	-	10	-	∞	-
nearest	-	1	-	1	-
distance	-	10	-	∞	-



```

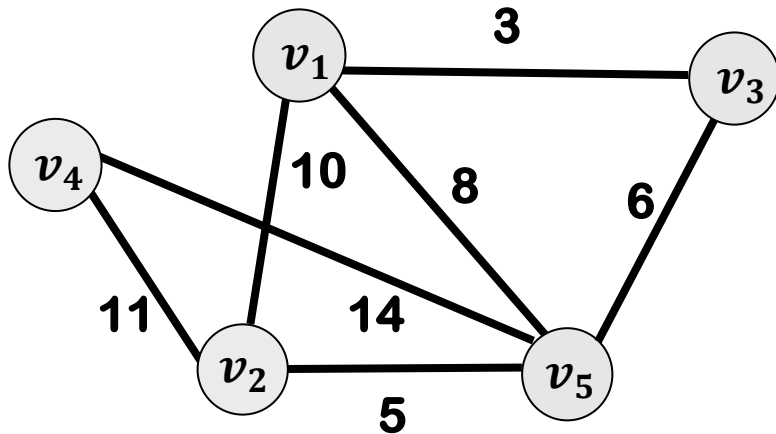
if(W[2][5] < distance[2]):
    distance[2] = W[2][5]
    nearest[2] = 5

```

```

if(W[4][5] < distance[4]):
    distance[4] = W[4][5]
    nearest[4] = 5

```



FOR EACH i in distance DO

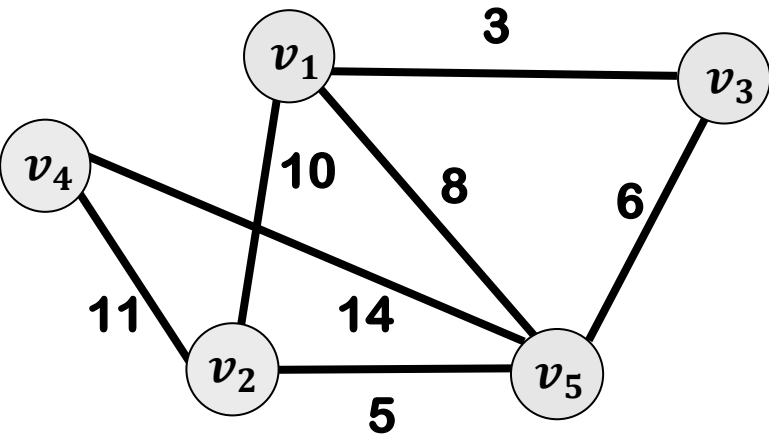
```

    IF  $W[i][b] < \text{distance}[i]$  THEN
         $\text{distance}[i] = W[i][b]$ 
         $\text{nearest}[i] = b$ 

```

vertices	1	2	3	4	5
nearest	-	1	-	1	-
distance	-	10	-	∞	-
nearest	-	1	-	1	-
distance	-	10	-	∞	-
nearest	-	5	-	5	-
distance	-	5	-	14	-

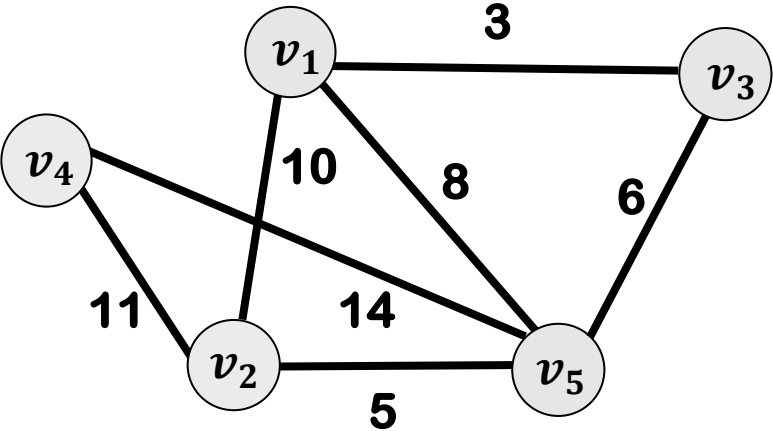

```
b=index of minimum element in distance
a=nearest[b]
F.add((a,b))
```



vertices	1	2	3	4	5
nearest	-	1	-	1	-
distance	-	10	-	∞	-
nearest	-	1	-	1	-
distance	-	10	-	∞	-
nearest	-	5	-	5	-
distance	-	5	-	14	-

b=2
a=5
F={(3,1),(3,5),(2,5)}

`distance.delete(b)`
`nearest.delete(b)`

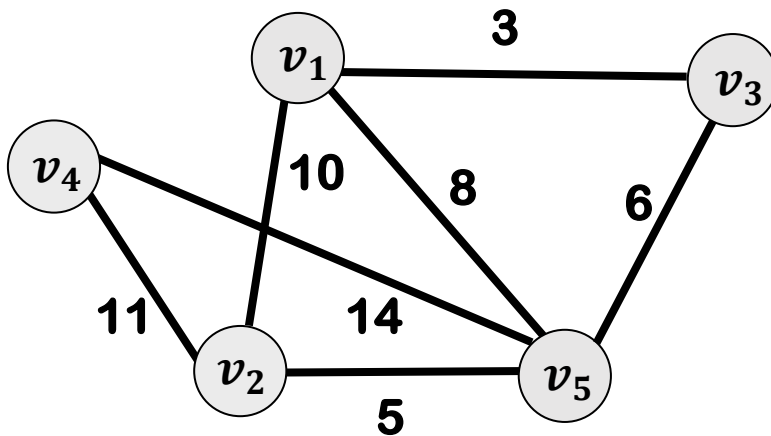


vertices	1	2	3	4	5
nearest	-	-	-	1	-
distance	-	-	-	∞	-
nearest	-	-	-	1	-
distance	-	-	-	∞	-
nearest	-	-	-	5	-
distance	-	-	-	14	-

FOR EACH i in distance DO

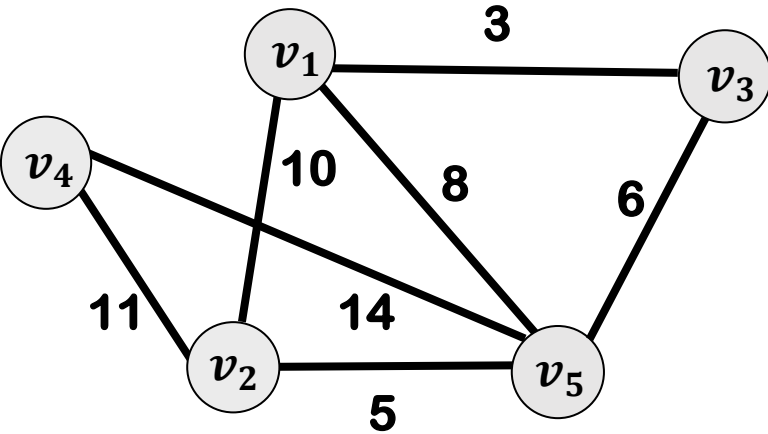
IF $W[i][b] < \text{distance}[i]$ THEN
 $\text{distance}[i] = W[i][b]$
 $\text{nearest}[i] = b$

if($W[4][2] < \text{distance}[4]$)
 $\text{distance}[4] = W[4][2]$
 $\text{nearest}[4] = 2$



vertices	1	2	3	4	5
nearest	-	-	-	1	-
distance	-	-	-	∞	-
nearest	-	-	-	1	-
distance	-	-	-	∞	-
nearest	-	-	-	5	-
distance	-	-	-	14	-
nearest	-	-	-	2	-
distance	-	-	-	11	-

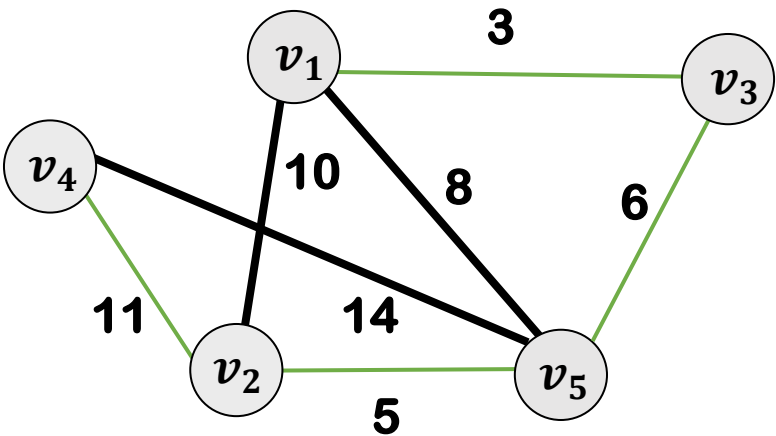
```
b=index of minimum element in distance
a=nearest[b]
F.add((a,b))
```



vertices	1	2	3	4	5
nearest	-	-	-	1	-
distance	-	-	-	∞	-
nearest	-	-	-	1	-
distance	-	-	-	∞	-
nearest	-	-	-	4	-
distance	-	-	-	14	-
nearest	-	-	-	2	-
distance	-	-	-	11	-

b=4
a=2
F={ (3,1), (3,5), (2,5), (2,4) }

distance.delete(b)
nearest.delete(b)



vertices	1	2	3	4	5
nearest	-	-	-	-	-
distance	-	-	-	-	-
nearest	-	-	-	-	-
distance	-	-	-	-	-
nearest	-	-	-	-	-
distance	-	-	-	-	-
nearest	-	-	-	-	-
distance	-	-	-	-	-
nearest	-	-	-	-	-
distance	-	-	-	-	-

$F = \{(3,1), (3,5), (2,5), (2,4)\}$

1. Let n be a positive integer. Write pseudocode to represent n as the sum of the maximum number of pairwise distinct positive integers. (Time complexity: $O(n)$)

Input. A positive integer n .

Output. The maximum k such that n can be represented as the sum $a_1 + \dots + a_k$ of k distinct positive integers. Also you need to output all integers $a_1, a_2, \dots, a_k$.

Input: $n=8$,

Output: return 3 and 1,2,5

$8=1+2+5$

Pick the **smallest unused positive integer** (starting from 1),

Input: $n=17$

Output: return 5 and 1,2,3,4,7

$17=1+2+3+4+7$

Input: $n=27$

Output: 6 and 1,2,3,4,5,12

$27=1+2+3+4+5+12$

1. Let n be a positive integer. Write pseudocode to represent n as the sum of the maximum number of pairwise distinct positive integers. (Time complexity: $O(n)$)

Input. A positive integer n .

Output. The maximum k such that n can be represented as the sum $a_1 + \dots + a_k$ of k distinct positive integers. Also you need to output all integers $a_1, a_2, \dots, a_k$.

Example: $n=10$

$n=10$
 $\text{sum}=0$

0	1	2	3	4	5	6

1. Let n be a positive integer. Write pseudocode to represent n as the sum of the maximum number of pairwise distinct positive integers. (Time complexity: $O(n)$)

Input. A positive integer n .

Output. The maximum k such that n can be represented as the sum $a_1 + \dots + a_k$ of k distinct positive integers. Also you need to output all integers $a_1, a_2, \dots, a_k$.

Example: $n=10$

$n=10$
 $\text{sum}=1$

0	1	2	3	4	5	6
1						

1. Let n be a positive integer. Write pseudocode to represent n as the sum of the maximum number of pairwise distinct positive integers. (Time complexity: $O(n)$)

Input. A positive integer n .

Output. The maximum k such that n can be represented as the sum $a_1 + \dots + a_k$ of k distinct positive integers. Also you need to output all integers $a_1, a_2, \dots, a_k$.

Example: $n=10$

$n=10$
 $\text{sum}=3$

0	1	2	3	4	5	6
1	2					

1. Let n be a positive integer. Write pseudocode to represent n as the sum of the maximum number of pairwise distinct positive integers. (Time complexity: $O(n)$)

Input. A positive integer n .

Output. The maximum k such that n can be represented as the sum $a_1 + \dots + a_k$ of k distinct positive integers. Also you need to output all integers $a_1, a_2, \dots, a_k$.

Example: $n=10$

$n=10$
 $\text{sum}=6$

0	1	2	3	4	5	6
1	2	3				

1. Let n be a positive integer. Write pseudocode to represent n as the sum of the maximum number of pairwise distinct positive integers. (Time complexity: $O(n)$)

Input. A positive integer n .

Output. The maximum k such that n can be represented as the sum $a_1 + \dots + a_k$ of k distinct positive integers. Also you need to output all integers $a_1, a_2, \dots, a_k$.

Example: $n=10$

$n=10$

sum=10

0	1	2	3	4	5	6
1	2	3	4			

1. Let n be a positive integer. Write pseudocode to represent n as the sum of the maximum number of pairwise distinct positive integers. (Time complexity: $O(n)$)

Input. A positive integer n .

Output. The maximum k such that n can be represented as the sum $a_1 + \dots + a_k$ of k distinct positive integers. Also you need to output all integers $a_1, a_2, \dots, a_k$.

Example: $n=8$

$n=8$
 $\text{sum}=0$

0	1	2	3	4	5	6

1. Let n be a positive integer. Write pseudocode to represent n as the sum of the maximum number of pairwise distinct positive integers. (Time complexity: $O(n)$)

Input. A positive integer n .

Output. The maximum k such that n can be represented as the sum $a_1 + \dots + a_k$ of k distinct positive integers. Also you need to output all integers $a_1, a_2, \dots, a_k$.

Example: $n=8$

$n=8$

sum=1

0	1	2	3	4	5	6
1						

1. Let n be a positive integer. Write pseudocode to represent n as the sum of the maximum number of pairwise distinct positive integers. (Time complexity: $O(n)$)

Input. A positive integer n .

Output. The maximum k such that n can be represented as the sum $a_1 + \dots + a_k$ of k distinct positive integers. Also you need to output all integers $a_1, a_2, \dots, a_k$.

Example: $n=8$

$n=8$

sum=3

0	1	2	3	4	5	6
1	2					

1. Let n be a positive integer. Write pseudocode to represent n as the sum of the maximum number of pairwise distinct positive integers. (Time complexity: $O(n)$)

Input. A positive integer n .

Output. The maximum k such that n can be represented as the sum $a_1 + \dots + a_k$ of k distinct positive integers. Also you need to output all integers $a_1, a_2, \dots, a_k$.

Example: $n=8$

$n=8$
 $\text{sum}=6$

0	1	2	3	4	5	6
1	2	3				

1. Let n be a positive integer. Write pseudocode to represent n as the sum of the maximum number of pairwise distinct positive integers. (Time complexity: $O(n)$)

Input. A positive integer n .

Output. The maximum k such that n can be represented as the sum $a_1 + \dots + a_k$ of k distinct positive integers. Also you need to output all integers $a_1, a_2, \dots, a_k$.

Example: $n=8$

$n=8$
 $\text{sum}=6$

0	1	2	3	4	5	6
1	2	3+2				

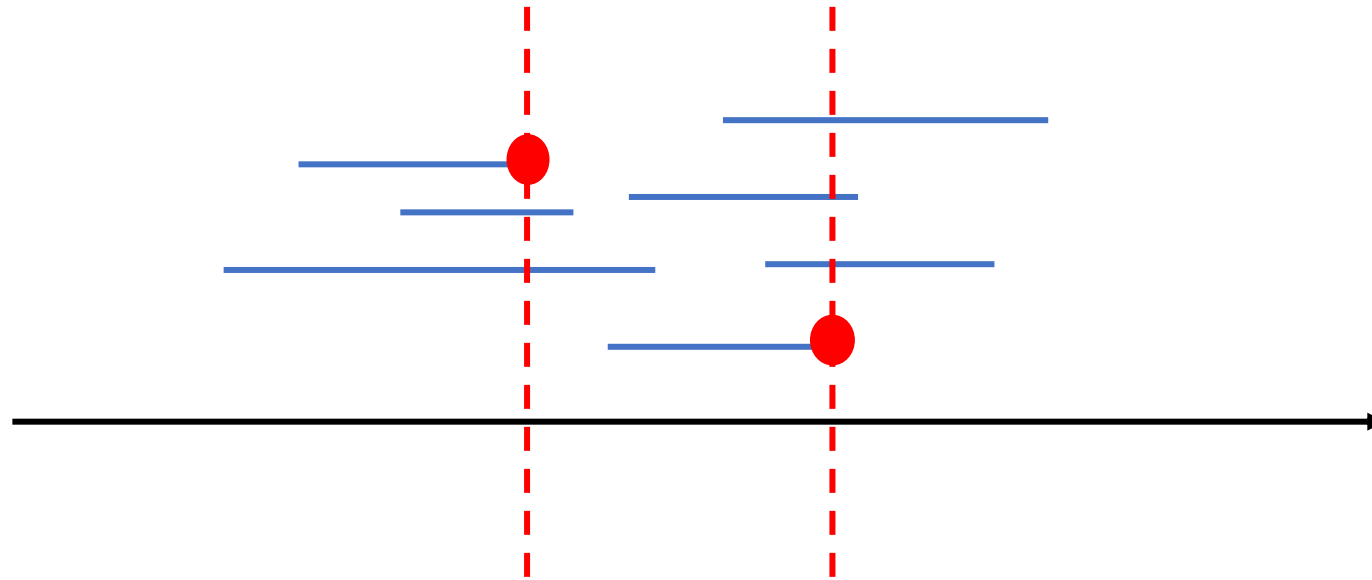
1. Let n be a positive integer. Write pseudocode to represent n as the sum of the maximum number of pairwise distinct positive integers. (Time complexity: $O(n)$)

Input. A positive integer n .

Output. The maximum k such that n can be represented as the sum $a_1 + \dots + a_k$ of k distinct positive integers. Also you need to output all integers $a_1, a_2, \dots, a_k$.

```
def MaxDistinctSum(n):  
    sum=0  
    k=1  
    result=[]  
    while sum + k <= n:  
        result.append(k)  
        sum= sum + k  
        k=k+1  
    result[-1]+= n - sum  
    return len(result),result
```

2. Covering Segments by Points Problem: We are given a sequence of n intervals $[l_1, r_1], \dots, [l_n, r_n]$ on x-axis. Find a set of points of minimum size such that each segment $[l_i, r_i]$ contains a point, i.e., there exists a point x from this set such that $l_i \leq x \leq r_i$, for all $1 \leq i \leq n$. In other words, find the minimum number of points on x-axis needed to cover all given intervals.



Solution:

First, we sort the intervals based on their right endpoint (r_i) in ascending order.

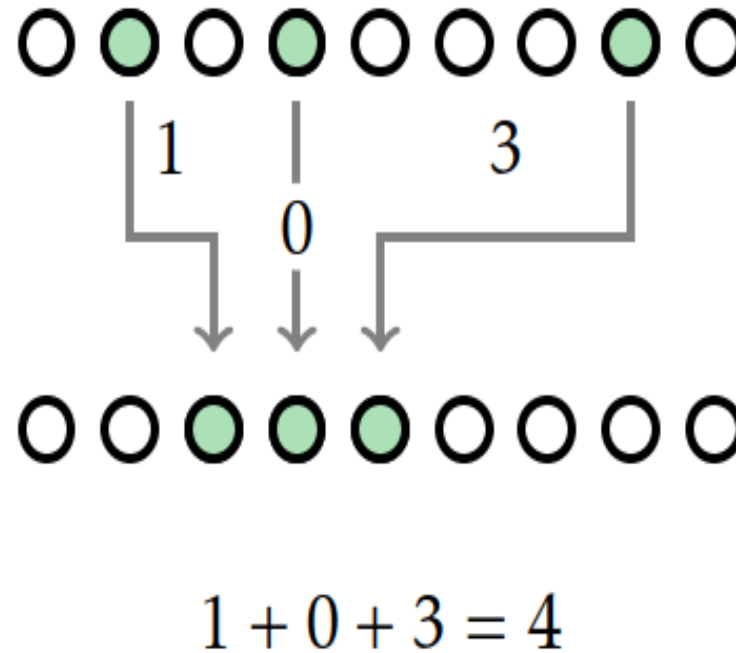
We follow a greedy strategy by choosing the rightmost endpoint that has not been covered by previously chosen points. To check if the last chosen point, say p , covers interval $[l_i, r_i]$ or not, we need to see if $p < l_i$. Sorting the intervals based on right endpoints ensures that we make the greedy choice at each step, resulting in the minimum number of points needed to cover all intervals.

```
minimum_cover(intervals):  
    sort intervals by right endpoint r_i in increasing order  
  
    S = empty set #output  
    p =  $-\infty$   
  
    for each interval (l_i, r_i):  
        if p < l_i:  
            p = r_i  
            add p to S  
  
    return S
```

3. Given positions of n friends in the first row of a theater, find the minimal number of moves to sit all friends together. In each move, a friend moves to a seat to the left or a seat to the right.

Input: A set of n distinct positive integers in the interval from 1 to N . This set represents positions of n friends that have bought individual (not necessarily adjacent) seats in the first row of a theater. Since all other seats in the first row turned out to be empty, the friends want to start moving to the neighboring seats (to the left or to the right) and continue until they all sit together in n consecutive chairs.

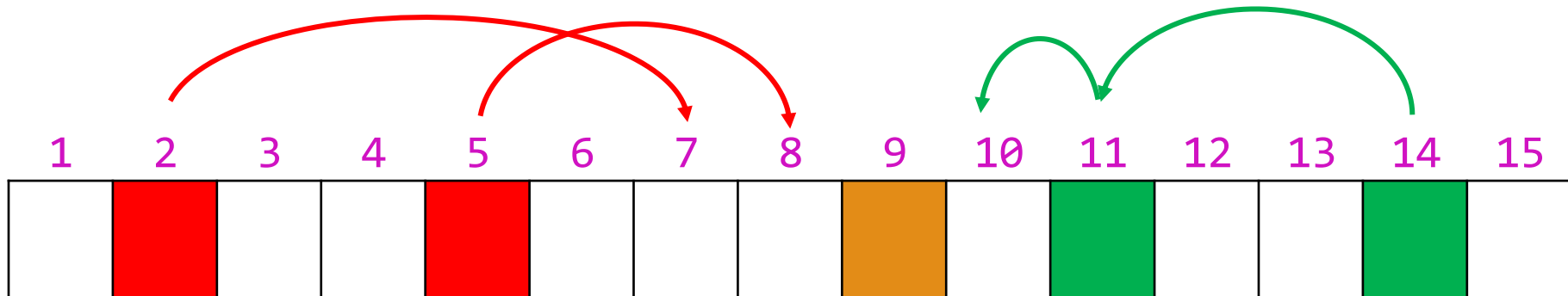
Output: Find the minimal number of moves to sit all friends together.



Solution: To design an optimal greedy algorithm, we sort the positions of friends in increasing order. To minimize the number of moves, we need to move all friends next to the person at the middle position. let P be an array of positions of friends. $P[i]$ = seat number of person $i+1$. See the following example:

They should move next to person at position 9. These are the moves:

- For the left side:
 - 5 to 8: 3 moves: $=9-5-1$
 - 2 to 7: 5 moves $=9-2-2$
- For the right side:
 - 11 to 10: 1 move $=11-9-1$
 - 14 to 11: 3 moves $=14-9-2$



P

2	5	9	11	14
---	---	---	----	----

```
def min_moves(P):  
    P.sort()  
    n = len(P)  
    m = n // 2    # median index  
    moves = 0  
    # left side  
    gap = 1  
    for i in range(m - 1, -1, -1):  
        moves += P[m] - P[i] - gap  
        gap += 1  
    # right side  
    gap = 1  
    for i in range(m + 1, n):  
        moves += P[i] - P[m] - gap  
        gap += 1  
  
    return moves
```

2	5	9	11	14
---	---	---	----	----

Any Question??

Have a nice day!